

NVIDIA Tesla y RTX GPUs: Arquitectura, CUDA y Optimización

George Briceño Celestino
Escuela de Ingeniería Electrónica
Instituto Tecnológico de Costa Rica
Cartago, Costa Rica
rousell@estudiantec.cr

Laura Elena Brenes Espinoza
Escuela de Ingeniería Electrónica
Instituto Tecnológico de Costa Rica
Alajuela, Costa Rica
lauritabrenes@estudiantec.cr

Marlon Méndez Naranjo
Escuela de Ingeniería Electrónica
Instituto Tecnológico de Costa Rica
San Carlos, Costa Rica
mnaranjo@estudiantec.cr

Rodrigo Venegas Mora
Escuela de Ingeniería Electrónica
Instituto Tecnológico de Costa Rica
Alajuela, Costa Rica
rovenegas@estudiantec.cr

I. INTRODUCCIÓN

Durante décadas, el incremento en el rendimiento de los procesadores de propósito general estuvo sustentado por la Ley de Moore y el escalamiento de Dennard. No obstante, la saturación física de este último y la aparición de la barrera de potencia (*power wall*) limitaron el escalado de frecuencias de reloj y la ganancia por paralelismo a nivel de instrucción (ILP). Mientras las CPUs tradicionales destinan la mayor parte de su silicio a estructuras complejas de control y extensas jerarquías de caché para minimizar la latencia de ejecución secuencial, las cargas de trabajo contemporáneas —como el cómputo científico, el procesamiento masivo de datos y el aprendizaje profundo— demandan un procesamiento orientado al ancho de banda computacional (*throughput*). En este contexto surge la computación heterogénea como el paradigma dominante: una arquitectura simbiótica *Host-Device* donde la CPU administra el flujo de control y las dependencias temporales, delegando las tareas intensivas en paralelismo de datos a aceleradores especializados basados en el modelo SIMT (*Single Instruction, Multiple Threads*), encarnados principalmente por las GPUs modernas [9]–[11].

Para satisfacer las demandas dispares del ecosistema tecnológico, el diseño arquitectónico de NVIDIA experimentó una bifurcación de objetivos, dando origen a dos líneas de procesamiento diferenciadas: la familia Tesla, hoy consolidada como la línea *Data Center*, y la familia RTX. La línea Tesla fue diseñada exclusivamente para dominar el cómputo de alto rendimiento (HPC) y la aceleración tensorial a gran escala. Para lograrlo, estas arquitecturas suprimen las interfaces físicas de salida de video, optimizando el área del silicio para integrar memoria de alto ancho de banda (HBM) y enlaces de interconexión de alta velocidad como NVLink, los cuales mitigan el cuello de botella del bus PCIe y permiten la comunicación masiva entre nodos en supercomputadoras. En contraste, la familia RTX fue concebida para satisfacer los requerimientos de estaciones de trabajo y gráficos de alta fidelidad. Esta

línea emplea memorias GDDR convencionales e incorpora hardware de propósito fijo, como los *RT Cores*, para acelerar el cálculo de intersecciones en el trazado de rayos (*Ray Tracing*), priorizando el renderizado visual complejo y la inferencia de inteligencia artificial a nivel de usuario [12]–[14].

La evolución generacional de estas arquitecturas evidencia una hiperespecialización del hardware orientada a satisfacer los requerimientos del cómputo masivo moderno. Estudios comparativos recientes de aceleradores de IA muestran un escalamiento dramático en la densidad de integración: mientras que la arquitectura Ampere (A100) inició este ciclo con 54.2 mil millones de transistores, su sucesora Hopper escaló a 80 mil millones, integrando además el *Transformer Engine* para aceleración nativa en precisión FP8. Finalmente, análisis de microarquitectura evidencian que Blackwell representa un cambio de paradigma al romper el límite del retículo fotolitográfico. Esta última generación integra 208 mil millones de transistores empleando un diseño de empaquetado multinúcleo donde dos matrices se interconectan mediante la interfaz NV-HBI a 10 TB/s. Este diseño incorpora Tensor Cores de quinta generación con capacidad nativa FP4, logrando maximizar el *throughput* sin sacrificar precisión [15].

II. ARQUITECTURA: DE AMPERE A BLACKWELL

II-A. Ampere

La arquitectura Ampere estableció la base moderna de las GPU NVIDIA para cómputo acelerado. La A100 incorpora Tensor Cores de tercera generación, soporte para *structured sparsity* con patrón 2:4 y Multi-Instance GPU (MIG), que aísla recursos de cómputo, caché y memoria HBM en instancias independientes [16]. En la línea GeForce y de estaciones de trabajo, GA102 implementa Ampere con un rendimiento FP32 duplicado respecto a la generación anterior y utiliza memoria GDDR6X con tasas de transferencia de hasta 19.5 Gbps [17].

La Figura 1 complementa la descripción de Ampere mostrando la organización interna de un SM: cuatro particiones, cada una con su Warp Scheduler, Dispatch Unit, Re-



Figura 1. Organización de un Streaming Multiprocessor (SM) de la arquitectura Ampere GA100.

gister File de $16,384 \times 32$ -bit, Tensor Core y unidades FP32/FP64/INT32. La caché L1 de datos y memoria compartida es de 192 KB. Adaptado de NVIDIA A100 Tensor Core GPU architecture [16].

II-B. Hopper

Hopper, representada por H100 y H200, intensifica el enfoque en IA generativa y cómputo a escala de centro de datos. La H100 incorpora NVLink de cuarta generación, memoria HBM3, Tensor Cores de cuarta generación y la Transformer Engine, que acelera modelos de lenguaje de gran escala mediante precisión FP8 dinámica. Además, mantiene MIG de segunda generación, permitiendo particionar la GPU en hasta siete instancias aisladas y escalando desde cargas individuales hasta sistemas multi-GPU y multi-nodo [18].

La Figura 2 complementa la descripción de Hopper mostrando el diagrama de bloques de la H100: 80 mil millones de transistores en TSMC 4N, 132 SMs con 2x rendimiento por ciclo, Tensor Cores de cuarta generación, HBM3, caché L2 de 50 MB, NVLink de cuarta generación (900 GB/s), PCIe Gen5 y MIG de segunda generación. Adaptado de NVIDIA H100 Tensor Core GPU architecture [18].

II-C. Blackwell

Blackwell continúa esta tendencia mediante mayor densidad de cómputo, Tensor Cores de quinta generación y soporte para formatos de precisión extremadamente baja, incluido FP4/NVFP4. La familia de centro de datos emplea múltiples

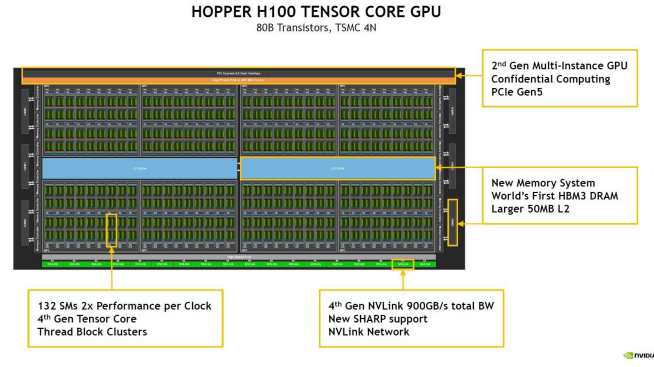


Figura 2. Diagrama de bloques de la GPU NVIDIA H100 basada en arquitectura Hopper.

dies interconectados mediante NV-HBI a 10 TB/s, presentándose ante CUDA como un único GPU programable [19]. En contraste, RTX Blackwell traslada las mejoras de IA, *neural shading* y *ray tracing* a las GPU GeForce, consolidando la convergencia entre aceleración de IA, gráficos y cómputo de propósito general [20].

La Figura 3 complementa la descripción de Blackwell mostrando la estructura interna de un SM: cuatro particiones con Register File de $16,384 \times 32$ -bit, Tensor Cores de quinta generación, unidades FP32/INT32, SFU y caché L1 de datos/memoria compartida de 128 KB. En la parte inferior se aprecia el RT Core de cuarta generación con motores para intersección de cajas, clústeres de triángulos, barrido lineal de esferas, micro-mapas de opacidad y compresión de clústeres de triángulos. Adaptado de NVIDIA Blackwell architecture [19].

III. MODELO DE PROGRAMACIÓN CUDA

CUDA expone el paralelismo de la GPU mediante una extensión de C/C++. Un *kernel* es una función ejecutada en el dispositivo por muchos hilos en paralelo; se declara con el especificador `__global__` y se invoca desde el host mediante la configuración de ejecución `<<grid, block>>`. Los hilos se organizan en *thread blocks*, que a su vez forman un *grid*; los hilos de un mismo bloque pueden cooperar mediante memoria compartida y sincronización [3].

IV. ECOSISTEMA DE DESARROLLO

IV-A. Modelo de Programación CUDA

El modelo de programación CUDA asume un sistema de cómputo heterogéneo compuesto por una CPU (*host*) y una GPU (*device*), cada una con su propia memoria [3]. El flujo de ejecución típico sigue estos pasos [3]:

1. Copiar datos de la memoria principal (*host*) a la memoria de la GPU (*device*).
2. La CPU lanza un *kernel* de cómputo en la GPU.
3. Los CUDA cores de la GPU ejecutan el *kernel* en paralelo.
4. Copiar los resultados de la memoria del *device* a la memoria del *host*.

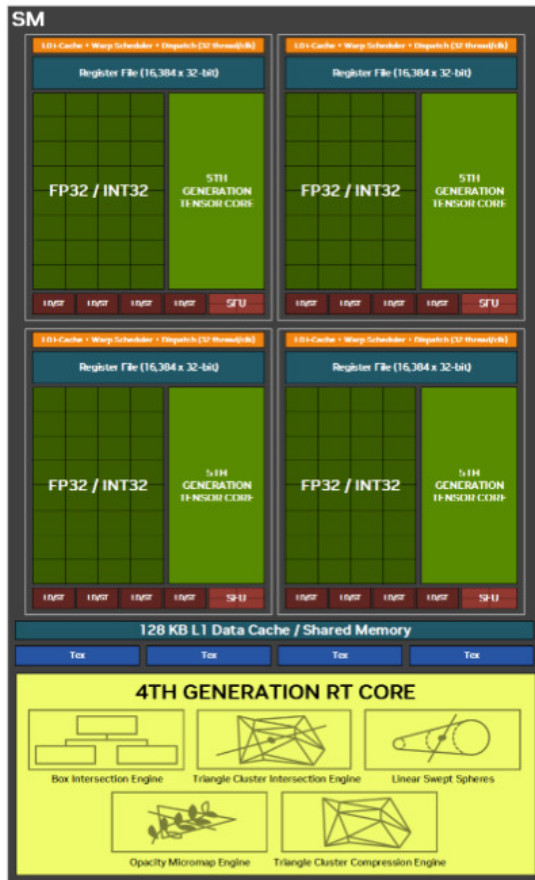


Figura 3. Estructura interna de un SM y RT Core de cuarta generación en Blackwell.

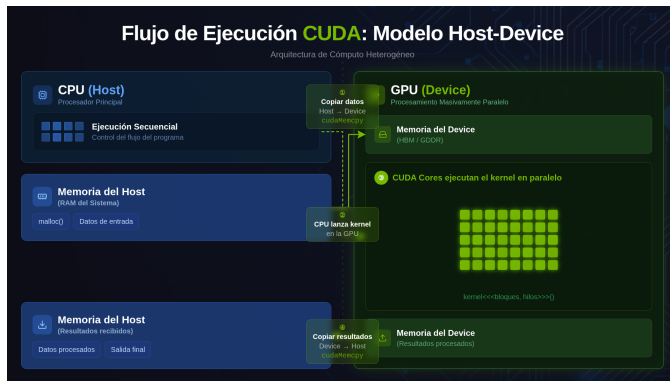


Figura 4. Flujo de ejecución CUDA bajo el modelo Host-Device.

IV-A1. Jerarquía de Threads: Los *threads* se organizan en *thread blocks* que forman un *grid* (1D, 2D o 3D); cada *block* admite hasta 1,024 *threads* [3]. Variables integradas — *threadIdx*, *blockIdx*, *blockDim*, *gridDim*— ubican a cada *thread*. A nivel de hardware, 32 *threads* forman un *warp* que ejecuta instrucciones SIMT en un Streaming Multiprocessor (SM) [3].

La jerarquía de memoria CUDA comprende registros (privados por *thread*), *shared memory* (por *block*, baja latencia),

cachés L1/L2 en chip y memoria global (HBM/GDDR) de mayor latencia [3].

IV-B. Ejemplo de Código: Adición de Vectores

El siguiente ejemplo muestra un programa completo en CUDA C++ que suma dos vectores de $N = 1024$ elementos. El *kernel* *vecAdd* se ejecuta en paralelo en la GPU, donde cada *thread* calcula la suma de un par de elementos [3].

```
#include <stdio.h>

// Kernel: cada thread suma un elemento
__global__ void vecAdd(float* A, float* B, float* C, int N)
{
    int i = blockDim.x * blockIdx.x + threadIdx.x;
    if (i < N) {
        C[i] = A[i] + B[i];
    }
}

int main() {
    int N = 1024;
    size_t size = N * sizeof(float);

    // Punteros del host
    float *h_A = (float*)malloc(size);
    float *h_B = (float*)malloc(size);
    float *h_C = (float*)malloc(size);

    // Inicializar vectores
    for (int i = 0; i < N; i++) {
        h_A[i] = 1.0f;
        h_B[i] = 2.0f;
    }

    // Punteros del device (GPU)
    float *d_A, *d_B, *d_C;
    cudaMalloc(&d_A, size);
    cudaMalloc(&d_B, size);
    cudaMalloc(&d_C, size);

    // Copiar datos host -> device
    cudaMemcpy(d_A, h_A, size, cudaMemcpyHostToDevice);
    cudaMemcpy(d_B, h_B, size, cudaMemcpyHostToDevice);

    // Configurar y lanzar kernel
    int threadsPerBlock = 256;
    int blocksPerGrid = (N + threadsPerBlock - 1) /
        threadsPerBlock;
    vecAdd<<<blocksPerGrid, threadsPerBlock>>>>(d_A, d_B, d_C,
        N);

    // Sincronizar y copiar resultados device -> host
    cudaDeviceSynchronize();
    cudaMemcpy(h_C, d_C, size, cudaMemcpyDeviceToHost);

    // Verificar resultado
    printf("C[0] = %f, C[1023] = %f\n", h_C[0], h_C[1023]);

    // Liberar memoria
    cudaFree(d_A); cudaFree(d_B); cudaFree(d_C);
    free(h_A); free(h_B); free(h_C);

    return 0;
}
```

Listing 1. Adición de vectores en CUDA C++. Fuente: CUDA C++ Programming Guide [3].

El calificador `__global__` marca funciones que corren en la GPU y se invocan desde la CPU; `blockDim.x*blockIdx.x+threadIdx.x` da el índice global de cada *thread*. `cudaMalloc`, `cudaMemcpy` y `cudaFree` gestionan la memoria del *device*, y `cudaDeviceSynchronize()` garantiza que el *kernel* termine antes de leer resultados [3].

Tabla I
CAPAS DEL ECOSISTEMA DE SOFTWARE CUDA.

Capa	Componentes	Descripción
Aplicaciones Frameworks	PyTorch, TensorFlow, JAX	Frameworks de alto nivel para ML/DL que traducen internamente las operaciones en llamadas CUDA [1].
CUDA-X Librerías	cuBLAS, cuDNN, TensorRT, NCCL, Thrust, cuFFT, CUTLASS	Bibliotecas optimizadas por dominio [4].
CUDA Runtime API	cudaMalloc, cudaMemcpy, «<>»	API de alto nivel para gestión de memoria, lanzamiento de kernels y sincronización [3].
CUDA Driver API	cuCtxCreate, cuModuleLoad	API de bajo nivel para control granular del contexto de la GPU y carga de módulos [2].
Compilador (NVCC)	NVCC → PTX → SASS	Compila CUDA C++ en código PTX intermedio y luego en instrucciones nativas de la GPU (SASS) [2].
Hardware de la GPU	CUDA Cores, Tensor Cores, SMs, HBM/GDDR	Los Streaming Multiprocessors ejecutan warps de 32 threads; los Tensor Cores aceleran operaciones matriciales [3].

Tabla II
PRINCIPALES BIBLIOTECAS DEL ECOSISTEMA CUDA-X [4].

Biblioteca	Dominio	Descripción
cuBLAS	Álgebra Lineal	Implementación de BLAS (Basic Linear Algebra Subprograms) acelerada en GPU.
cuDNN	Aprendizaje Profundo	Primitivas optimizadas para redes neuronales: convoluciones, normalización, activaciones y RNNs.
TensorRT, TensorRT-LLM	Inferencia	Optimizador e infraestructura de inferencia de alto rendimiento; soporta FP32, FP16, BF16, FP8 e INT8; optimizaciones especializadas para Transformers y LLMs.
NCCL	Comunicación Multi-GPU	Biblioteca de código abierto para comunicación colectiva de baja latencia y alto ancho de banda entre múltiples GPU y nodos.
Thrust	Algoritmos Paralelos	Biblioteca de plantillas C++ basada en la STL para algoritmos paralelos de alto nivel (<i>sort</i> , <i>scan</i> , <i>reduce</i>).

IV-C. Capas del Ecosistema de Software de NVIDIA

El ecosistema CUDA se organiza en capas jerárquicas, desde las aplicaciones de alto nivel hasta el hardware [4]. La Tabla I resume esta estructura.

IV-D. Principales Bibliotecas CUDA-X

Las bibliotecas NVIDIA CUDA-X ofrecen implementaciones altamente optimizadas que superan ampliamente a las alternativas basadas únicamente en CPU [4]. La Tabla II presenta las bibliotecas más relevantes.

IV-E. Herramientas de Desarrollo y Perfilado

El CUDA Toolkit incluye herramientas esenciales para el ciclo completo de desarrollo, depuración y optimización [2]:

- **NVCC:** compilador de CUDA C++ que genera código PTX e instrucciones SASS nativas.
- **Nsight Systems:** perfilador a nivel de sistema para el análisis de rendimiento CPU+GPU.
- **Nsight Compute:** perfilador detallado a nivel de kernel para el análisis de métricas de la GPU.
- **CUDA-GDB:** depurador de código CUDA en Linux.

IV-F. Lenguajes Soportados y Métodos de Acceso

CUDA es accesible desde múltiples lenguajes de programación y paradigmas [1], [5]:

- **CUDA C/C++:** extensión nativa del lenguaje, compilada con NVCC (basado en LLVM) o Clang.
- **CUDA Python 1.0:** desde CUDA 13.3, Python es un lenguaje oficialmente soportado para acceder a toda la plataforma CUDA [5].
- **Frameworks de alto nivel:** PyTorch, TensorFlow y JAX traducen automáticamente las operaciones en llamadas CUDA, permitiendo a los usuarios aprovechar la GPU sin programar directamente en CUDA C++ [1].

IV-G. Patrones de Optimización

Optimizar una aplicación CUDA implica balancear unidades de cómputo, jerarquía de memoria, ancho de banda y sobrecarga de lanzamiento. Las guías de NVIDIA para Ampere y Blackwell coinciden en tres prácticas base: configuración de lanzamiento, accesos coalescidos a memoria global y evitar accesos redundantes [6], [7].

IV-G1. Coalescencia de Memoria Global: Cuando los hilos de un *warp* leen direcciones contiguas, el hardware agrupa sus peticiones en unas pocas transacciones. Cuando no, cada hilo paga su propio viaje a memoria. Esa diferencia, que en el papel parece menor, suele decidir si un *kernel* es rápido o lento. Los patrones con *strides* no unitarios, accesos dispersos o lecturas repetidas castigan al subsistema de memoria y terminan convirtiéndose en el cuello de botella principal. Un caso típico es un *warp* que recorre una matriz por columnas cuando está almacenada por filas. La solución habitual pasa por cargar los datos a memoria compartida, reordenarlos ahí y dejar que cada hilo los lea después con un patrón favorable [6].

IV-G2. GEMM por Teselas: Las multiplicaciones de matrices densas concentran la mayor parte del trabajo en muchas cargas de aprendizaje profundo y visión por computadora. El enfoque *tile-based* parte los operandos en bloques que caben en memoria compartida o en registros, de modo que los datos se reutilicen entre hilos y se reduzcan los accesos repetidos a memoria global. En la práctica, esto se implementa con bibliotecas especializadas: cuBLAS, orientada a primitivas de álgebra lineal de alto rendimiento, o CUTLASS, que ofrece plantillas C++ y un DSL en Python para construir *kernels* GEMM adaptados a Tensor Cores [4], [6].

IV-G3. Fusión de Kernels: Si una convolución va seguida de una ReLU, escribir el resultado intermedio en memoria global para volver a leerlo es puro desperdicio. La fusión de *kernels* evita justo eso: combina varias operaciones en un solo *kernel*, se salta los buffers intermedios y reduce los lanzamientos desde el *host*. TensorRT lo aplica solo cuando detecta patrones compatibles —convolución más activación, cadenas elemento a elemento— y el resultado conserva la semántica de la red original [8].

IV-G4. Batching y CUDA Graphs: El *batching* procesa múltiples entradas a la vez, maximizando el trabajo útil por *kernel*; en TensorRT, lotes mayores casi siempre mejoran la

eficiencia en GPU [8]. CUDA Graphs capturan una secuencia completa de lanzamientos en un grafo reutilizable, reduciendo la sobrecarga de CPU en *pipelines* con topologías fijas [8].

V. CONCLUSIONES

La evolución de las GPU NVIDIA evidencia una clara tendencia hacia la **hiperespecialización del hardware**. La línea Tesla, actualmente orientada al segmento *Data Center*, prioriza aplicaciones de cómputo de alto rendimiento e inteligencia artificial, mientras que la familia RTX mantiene un enfoque dirigido principalmente a gráficos, estaciones de trabajo y aceleración de IA local. A pesar de estas diferencias, ambas familias han convergido progresivamente en el uso de tecnologías como los **Tensor Cores**, demostrando una integración cada vez mayor entre gráficos, inteligencia artificial y cómputo de propósito general.

Asimismo, el desarrollo de las arquitecturas **Ampere, Hopper y Blackwell** muestra un salto generacional continuo. Cada generación ha incorporado mejoras en aspectos como el **ancho de banda de memoria, la precisión numérica, la capacidad de procesamiento tensorial y el aislamiento de recursos**. Ampere consolidó tecnologías como MIG y la *sparsity* estructurada, Hopper introdujo el *Transformer Engine* y el uso de FP8, mientras que Blackwell amplía esta tendencia mediante soporte para FP4/NVFP4 y arquitecturas *multi-die*.

Otro aspecto fundamental es el crecimiento de **CUDA como un ecosistema completo de desarrollo**. CUDA no se limita únicamente a la programación de *kernels* en C/C++, sino que incluye bibliotecas especializadas, herramientas de perfilado, *frameworks* de alto nivel y soporte para otros lenguajes. La incorporación de CUDA Python reduce la barrera de entrada y facilita que más desarrolladores puedan utilizar la capacidad de procesamiento de las GPU sin depender exclusivamente de programación CUDA C++ de bajo nivel.

Finalmente, el análisis de los patrones de optimización demuestra que el rendimiento de una aplicación en GPU depende de un adecuado **balance entre capacidad de cómputo, ancho de banda de memoria y sobrecarga de ejecución**. Técnicas como los accesos coalescidos, el uso de memoria compartida, el *tiling*, la fusión de *kernels*, el *batching* y CUDA Graphs permiten utilizar de forma más eficiente los recursos disponibles. Por lo tanto, obtener un alto rendimiento no depende únicamente de disponer de una GPU más potente, sino también de diseñar el software de manera que aproveche correctamente su arquitectura.

REFERENCIAS

- [1] NVIDIA Corporation, "CUDA Platform for Accelerated Computing," 2026. [Online]. Available: <https://developer.nvidia.com/cuda>
- [2] NVIDIA Corporation, "CUDA Toolkit Documentation," 2026. [Online]. Available: <https://docs.nvidia.com/cuda/>
- [3] NVIDIA Corporation, "CUDA C++ Programming Guide," 2026. [Online]. Available: <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>
- [4] NVIDIA Corporation, "NVIDIA CUDA-X Libraries," 2026. [Online]. Available: <https://developer.nvidia.com/gpu-accelerated-libraries>
- [5] NVIDIA Developer Blog, "CUDA Python 1.0: Stable APIs, One Foundation, Full Platform Access," 2026. [Online]. Available: <https://developer.nvidia.com/blog/unifying-the-cuda-python-ecosystem/>
- [6] NVIDIA Corporation, "NVIDIA Ampere GPU Architecture Tuning Guide," 2026. [Online]. Available: <https://docs.nvidia.com/cuda/ampere-tuning-guide/>
- [7] NVIDIA Corporation, "NVIDIA Blackwell GPU Architecture Tuning Guide," 2026. [Online]. Available: <https://docs.nvidia.com/cuda/blackwell-tuning-guide/>
- [8] NVIDIA Corporation, "Optimizing TensorRT Performance," 2026. [Online]. Available: <https://docs.nvidia.com/deeplearning/tensorrt/latest/performance/optimization.html>
- [9] J. L. Hennessy and D. A. Patterson, "A New Golden Age for Computer Architecture," *Communications of the ACM*, vol. 62, no. 2, pp. 48–60, Feb. 2019. doi: 10.1145/3282307. [Online]. Available: <https://cacm.acm.org/research/a-new-golden-age-for-computer-architecture/>
- [10] NVIDIA Corporation, "Introduction — CUDA Programming Guide," 2026. [Online]. Available: <https://docs.nvidia.com/cuda/cuda-programming-guide/01-introduction/introduction.html>
- [11] J. Shalf, "The Future of Computing Beyond Moore's Law," *Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences*, vol. 378, no. 2166, Art. no. 20190061, 2020. doi: 10.1098/rsta.2019.0061. [Online]. Available: <https://royalsocietypublishing.org/rsta/article/378/2166/20190061/111562/The-future-of-computing-beyond-Moore-s>
- [12] NVIDIA Corporation, "NVIDIA Tesla V100 Tensor Core GPU," [Online]. Available: <https://www.nvidia.com/es-la/data-center/tesla-v100/>
- [13] NVIDIA Corporation, "NVIDIA Tesla P100: The Most Advanced Datacenter Accelerator Ever Built — Featuring Pascal GP100, the World's Fastest GPU," White Paper, 2016. [Online]. Available: <https://images.nvidia.com/content/pdf/tesla/whitepaper/pascal-architecture-whitepaper.pdf>
- [14] NVIDIA Corporation, "NVIDIA Turing GPU Architecture," White Paper, 2018. [Online]. Available: <https://www.nvidia.com/content/dam/en-zz/Solutions/design-visualization/technologies/turing-architecture/NVIDIA-Turing-Architecture-Whitepaper.pdf>
- [15] A. Elgaber and B. A. Slisal, "The Evolution of NVIDIA AI Accelerators and Their Impact on Artificial Intelligence Model Performance: A Comparative Analysis of NVIDIA A100, H100, H200, and Blackwell Architectures," *Comprehensive Journal of Science*, vol. 11, no. 42, pp. 702–719, 2026. doi: 10.65405/95b0ew04. [Online]. Available: https://www.researchgate.net/publication/414568614_The_Evolution_of_NVIDIA_AI_Accelerators_and_Their_Impact_on_Artificial_Intelligence_Model_Performance_A_Comparative_Analysis_of_NVIDIA_A100_H100_H200_and_Blackwell_Architectures
- [16] NVIDIA Corporation, "NVIDIA A100 Tensor Core GPU Architecture," White Paper, 2020. [Online]. Available: <https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/nvidia-ampere-architecture-whitepaper.pdf>
- [17] NVIDIA Corporation, "NVIDIA Ampere GA102 GPU Architecture," White Paper, 2020. [Online]. Available: <https://www.nvidia.com/content/PDF/nvidia-ampere-ga-102-gpu-architecture-whitepaper-v2.pdf>
- [18] NVIDIA Corporation, "NVIDIA H100 Tensor Core GPU Architecture," White Paper, 2022. [Online]. Available: <https://resources.nvidia.com/en-us-tensor-core>
- [19] NVIDIA Corporation, "NVIDIA Blackwell Architecture," White Paper, 2024. [Online]. Available: <https://resources.nvidia.com/en-us-blackwell-architecture>
- [20] NVIDIA Corporation, "NVIDIA RTX Blackwell GPU Architecture," 2024. [Online]. Available: <https://www.nvidia.com/en-us/geforce/graphics-cards/50-series/rtx-50-series/>